AI usage log

Per the Week 1 AI Usage Log template: log meaningful AI interactions — architecture, generated modules, debugging sessions — updated as work happens, not reconstructed after the fact.

Date	Task / component	Tool & model	What I asked for	What I verified or changed	What I built myself
2026-07-15	Architecture design (ingestion + chat pipeline, backend layering, Postgres schema/ERD, pattern-justification rationale for chunking/retrieval/embedding/generation/DB choices)	Claude Code (Sonnet)	A design doc and architecture for a RAG chatbot meeting the Week 1 brief's functional requirements and technical constraints	Reviewed the generated architecture against the brief line by line; cross-checked stack choices (Azure AI Search, Foundry models, local Postgres) against my actual Azure account access before adopting them, since the brief's default assumptions (Azure-hosted Postgres) didn't match what I actually had provisioned	The decision to deviate from Azure-hosted Postgres to local Docker Postgres, and the call to document that deviation explicitly rather than quietly work around it
2026-07-15	Design doc, cost-estimate and retrieval-quality-note templates (structure only)	Claude Code (Sonnet)	Scaffolding for the cost estimate and retrieval-quality docs so the sections existed before real numbers did	Confirmed these were structural placeholders, not final content — flagged for myself to fill in with real measured numbers once ingestion/chat were actually running, not before	The plan to measure real numbers before writing the docs, instead of estimating them upfront
2026-07-16	Backend implementation (routers/services/repositories, SQLAlchemy models, Alembic migration, Docker Compose, ingestion pipeline: content-sniffing loader, structure-aware chunker, embedding+indexing)	Claude Code (Sonnet)	A working backend implementing the layered architecture from the design doc, plus an ingestion pipeline for the 20-doc corpus	Ran the loader/chunker against all 20 real corpus files before any Azure calls, not just read the code; executed a full live ingestion run (1,023 chunks indexed) against real Azure Search credentials; spot-checked retrieval with 10 real questions. Caught two real bugs this way: the refusal threshold was calibrated for the wrong score scale (assumed 0-1, Azure's semantic reranker is ~0-4), and an interrupted ingestion run had orphaned stale Search index entries	Diagnosing both bugs from the actual symptom (wrong refusal behavior, stale index entries) rather than accepting the first fix offered, and deciding to commit per-document during ingestion to prevent the orphaning class of bug
2026-07-16	Azure Foundry integration (client wiring for GPT-5 generation, text-embedding-3-large embeddings)	Claude Code (Sonnet)	A Foundry client for chat generation and embeddings	The first attempt assumed the classic Azure OpenAI endpoint shape (Chat Completions API) — wrong for this resource. Corrected after I supplied the real endpoint, then verified live: an actual Responses API call, a real multi-turn history exchange, and a real embedding call, before accepting the client code	Catching that the initial endpoint assumption was wrong by actually hitting the API, instead of trusting the first plausible-looking client code
2026-07-17	Model selection + usage analytics (backend)	Claude Code (Sonnet)	Support for choosing between GPT-5, GPT-5.5, and DeepSeek V3.2 per request, plus real token-usage tracking per message	Verified live that gpt-5.5 and deepseek-v3.2 are both reachable through the same OpenAI-compatible client/endpoint as gpt-5 — no separate SDK route needed for DeepSeek, contradicting my Week 1 assumption in the design doc. Confirmed the model-validation fallback works by sending a request with `"model": "nonsense-model"` and checking it silently falls back to gpt-5 rather than erroring	Deciding to expose `/api/v1/chat/models` as a single source of truth for the frontend instead of hardcoding the model list twice
2026-07-17	Model selector, shared nav, admin analytics dashboard (frontend)	Claude Code (Sonnet)	A model-selector dropdown, a consistent sidebar across Chat/Admin, and an analytics dashboard in the admin view	Noticed the Chat/Admin nav links sat in different positions per page (bottom-left on chat, top-right on admin) — a real inconsistency, not a nitpick — and had it restructured into one shared Sidebar shell	Deciding that the conversation list needed to be its own component (`ConversationList`) so `Sidebar` itself could stay generic and reusable across both pages
2026-07-17	Scrollbar styling	Claude Code (Sonnet)	Scrollbars that match the dark theme instead of the default OS bar	Compared side-by-side against ChatGPT/Gemini's thin scrollbars and confirmed the new 8px transparent-track styling reads the same way in both Firefox and Chromium	—
2026-07-17	Analytics sync fix + dark/light theme	Claude Code (Sonnet)	A fix for admin analytics numbers not reconciling, plus a light theme to	Found the root cause myself: `assistant_message_count` included refused turns (saved with `model=NULL`) but `usage_by_model` filtered them out, so the two numbers could never match. Verified the fix by confirming	Deciding to restructure the model around `question_count = answered_count + refused_count` by construction, and to

Date	Task / component	Tool & model	What I asked for	What I verified or changed	What I built myself
			go with the existing dark one	`usage_by_model` 's counts now sum exactly to `answered_count`	surface the refusal count as its own stat tile since it's useful for demoing guardrail behavior
2026-07-17	Citation-dropping fix	Claude Code (Sonnet)	A fix for citations still showing on answers the model itself refused	Reproduced the bug myself first: asked "Hello there what is anthropic," which retrieved boilerplate scoring 2.02 (just over the 2.0 refusal threshold) so retrieval technically "passed," even though the model correctly said it didn't have the information. Verified the fix by re-running the same question and confirming citations no longer appear on a refusal	Recognizing that citations need to be reconciled against what the model's answer actually says, not attached purely from retrieval's pre-generation result
2026-07-17	Theme toggle position fix	Claude Code (Sonnet)	A fix for the theme toggle's position shifting between pages	Traced it to a layout issue, not a state bug: `Sidebar` 's footer sat below a conditionally-rendered spacer that only exists when children are passed, so `ChatView` (passes children) and `AdminView` (passes none) rendered the footer at different heights. Verified by moving `ThemeToggle` out of `Sidebar` entirely into `App.jsx` and confirming it now sits in the same fixed position on both routes	Diagnosing that the same component could legitimately render in two different places depending on its parent, rather than assuming it was a CSS specificity issue
2026-07-20	Citation UI redesign (Sources panel)	Claude Code (Sonnet)	A ChatGPT-style slide-in Sources panel to replace the always-expanded inline citation chips, plus shorter answers/snippets	Screenshotted the rebuilt frontend to confirm it rendered cleanly; could not click-test the panel interactively (no browser driver set up, declined installing one) — flagged this gap explicitly rather than claiming full visual verification	The decision to lift panel state to `ChatView` as a single shared overlay rather than per-message, matching how ChatGPT's own Sources panel behaves
2026-07-20	Citation accuracy bug (retirement vs. leave)	Claude Code (Sonnet)	Investigated a real report that an "employee retirement" question was citing unrelated "employee leave" content	Reproduced it myself against the live DB (found the actual conversation in Postgres, not a guess) — a borderline-scoring FMLA-leave chunk had cleared the retrieval threshold alongside a genuinely relevant 401(k) chunk, and all 5 retrieved chunks were shown as "sources" regardless of which ones the model's answer actually used	Tracing the bug to `chat.py` 's citation payload being built from raw retrieval results with no check against what the generated answer actually cited
2026-07-20	Citation filtering fix, then a second bug in my own fix	Claude Code (Sonnet)	A fix so only citations the answer actually grounds itself in are shown/saved	First version matched each citation's literal `[filename, chunk_ref]` string against the answer — verified live and found two bugs of my own: it missed cases where the model bundles several citations into one bracket (`[fileA, refA; fileB, refB]`), and a separate blunt "is this a refusal" regex was zeroing out citations on hedged-but-real answers. Both caught by testing real answers end-to-end, not by re-reading the code	Redesigning the check to match each bracket group for filename+chunk_ref co-occurring, and removing the separate refusal regex since correct per-citation filtering already reduces to nothing on a real refusal
2026-07-20	Speed/verbosity investigation	Claude Code (Sonnet)	Diagnosed why answers were slow and long despite using "powerful" models	Measured actual latency breakdown myself (retrieval vs. first-token vs. total) rather than assuming — found GPT-5 was silently spending ~28s on hidden reasoning tokens (4,593 total tokens for a ~150-word answer) with no `reasoning.effort` / `text.verbosity` param set. Tested each of the 3 models live against the real endpoint and found each accepts different parameter values (gpt-5.5 rejects "minimal", DeepSeek rejects the param entirely)	The per-model parameter map, after confirming live which values each model actually accepts rather than assuming they're uniform

Date	Task / component	Tool & model	What I asked for	What I verified or changed	What I built myself
2026-07-20	Corpus naming/content audit	Claude Code (Sonnet)	Was asked to name corpus documents accurately; opened and read every file's actual content instead of trusting filenames	Found `deo_handbook.pdf` and `wfahandbook.pdf` were federal-agency internal-operations manuals (hiring-examiner procedures; workforce-analysis methodology) with nothing to do with employee HR policy, making up 58% of the entire chunk index. Confirmed via `PdfReader` against the real files, not assumption from filenames	Recognizing that "naming" the documents correctly would surface a corpus-composition problem, and that fixing names alone wouldn't fix the underlying data-quality issue — flagged the removal decision to the owner rather than deciding unilaterally

| 2026-07-21 | Reindex blocking the whole server | Claude Code (Sonnet) | Asked why the site got slow/delayed whenever the admin re-index button was clicked | Traced it myself to two separate causes rather than accepting the first explanation: `POST /documents/reindex` awaited the full ~90s pipeline before responding, and the CPU-bound parse/chunk step ran directly on the event loop with no `await` point, freezing every other request (chat, doc list) for the whole process, not just the admin page. Verified by reading `ingestion_service.py`'s own code path, not guessing from symptoms | Deciding on the two-part fix: `BackgroundTasks` so the endpoint returns immediately, plus `asyncio.to_thread` around the parse/chunk step so it stops monopolizing the event loop | | 2026-07-21 | Free-tier cold start looking like lost data | Claude Code (Sonnet) | Asked why conversations/usage/documents periodically appeared to vanish, then reappear on their own | Recognized the pattern (temporary, self-resolving, not actual data loss) as Azure App Service Free-tier cold start, not a Postgres issue — confirmed by checking the App Service plan tier and noting Free/Shared tiers don't support Always On. Rejected a keep-alive-ping fix myself after checking F1's 60 CPU-minute/day quota — frequent pings risked a full daily outage, a worse failure mode than an occasional cold start | Deciding to fix the symptom on the frontend instead of the infra: retry-with-backoff on GET requests plus a "waking up the server" state, since Always On isn't available on this tier | | 2026-07-21 | Live incident: reindex hung for 15+ minutes | Claude Code (Sonnet) | Asked to check whether a reindex that had been running a long time was actually stuck | Verified by polling per-document `indexed_at` timestamps rather than trusting the run's own status field (which only writes `doc_count` / `chunk_count` at the very end) — confirmed zero forward progress for 9+ minutes against a normal 1.5-5 min run time. Reproduced every remaining document's parse/chunk step locally with timing to rule out a bad PDF (all finished in under 1.1s), narrowing it to a stuck network call with no timeout. Recovered live: restarted the App Service via `az webapp restart` and manually corrected the stuck `indexing_runs` row to `"failed"` via `psql`, since a hard process kill skips the code's own cleanup path | Diagnosing that the hang was in a live network call, not a parsing bug, by actually reproducing the remaining work locally instead of assuming | | 2026-07-21 | Corpus size reduction | Claude Code (Sonnet) | Asked to identify and remove the largest/least-necessary documents to speed up reindexing | Computed real chunk counts locally per file rather than guessing from file size alone — found `hr-manual.docx` was 32% of the entire corpus but only ever cited once in `retrieval-quality-note.md`, for a question `remote-work.docx` also answered. Recommended keeping `fmla-guide.pdf` despite its size (6.6MB) since it was independently validated as a correct citation, rather than cutting every large file indiscriminately | Deciding chunk-count-and-citation-history should drive removal decisions, not file size alone | | 2026-07-22 | Corpus replacement with a fictional "Contoso Corp" set | Claude Code (Sonnet) | Asked to wire in a new, evenly-formatted 20-document corpus the user had generated, flattening it out of nested subfolders | Verified every file parses before touching production: found all 5 new `.docx` files crashed `_load_docx` with `'NoneType' object has no attribute 'name'` (a paragraph's `python-docx` style was `None`) — reproduced locally first, confirmed the fix locally, then verified end-to-end against the local docker-compose stack (reindex + a real chat query) before deploying | Recognizing that `asyncio.gather` without `return_exceptions=True` meant this bug would have failed the *entire* reindex, not just 5 files, and rewriting the sniff-format tests to use synthetic fixtures instead of hardcoded corpus filenames so they can't break the same way again | | 2026-07-22 | Document pruning on reindex | Claude Code (Sonnet) | Asked why the document list still showed old file counts after files were deleted from the corpus and reindexed | Found `document_repo.py` had no delete/prune path — a reindex only ever upserts files it finds on disk, never revisiting a file that's gone. Checked `citations.document_id`'s foreign key before choosing an approach: it's non-cascading, so a hard `DELETE` would fail against any document with real citation history | Choosing soft-delete (`status = "removed"`, excluded from `/documents`) plus explicit Azure Search chunk deletion by `document_id` filter, over a schema migration to add `ON DELETE CASCADE`, since cascading would silently erase citations on old conversations | | 2026-07-22 | Corpus expansion to 24 documents | Claude Code (Sonnet) | Asked to push and deploy 4 new documents the user had added directly to the corpus (`CON-FIN-003`, `CON-HR-014`, `CON-HR-015`, `CON-OPS-002`) | Deployed, then triggered a reindex that suspiciously completed with the *old* counts (20 docs/179 chunks) instead of 24/222 despite a "successful" GH Actions run and a 200 health check. Diagnosed a real deployment propagation delay — Azure hadn't finished syncing the new files to disk yet when I reindexed — by checking the full run history's timestamps rather than trusting one result, and confirmed a second reindex ~90s later picked up all 24 correctly | Deciding not to trust a single "completed" reindex result at face value after a deploy, and re-verifying with a fresh reindex plus a live chat query against one of the new documents before calling it done | | 2026-07-22 | Frontend/backend QA pass via headless Chrome | Claude Code (Sonnet) | Asked to check for frontend issues (animations, indexing) and backend issues, using docker-compose for testing | Drove the actual running app via the Chrome DevTools Protocol rather than just reading component code: a real typed chat message through full streaming, the Admin re-index button click-through, Sources panel, and theme toggle, capturing console/network/exceptions at each step. Found real console output was clean throughout, but found a genuine gap: `pollUntilRunFinishes`'s `while (true)` polling loop has no timeout, so a repeat of the hung-reindex incident would leave the admin UI stuck polling forever with no error shown | Choosing to verify UI behavior by actually driving a browser instead of asserting from source reading alone, since a previous AI usage log entry (2026-07-20, Sources panel) had explicitly flagged not having done this as a gap |

Note on 2026-07-17/07-20 entries: drafted from commit messages and session actions, which already contain verification specifics — review before the demo and adjust "what I verified" / "what I built myself" to match your own account, since these are the columns most likely to come up in Friday Q&A. **The same applies to every entry above** — these are drafted from this session's actual commits and live debugging output, not reconstructed from memory, but review and restate them in your own words before the demo.